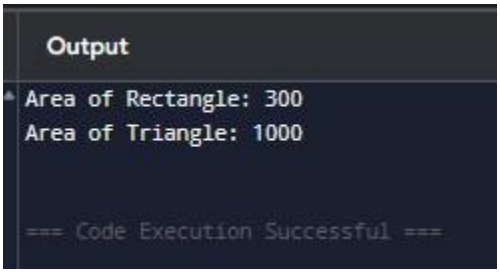
ASSIGNMENT NO. 7

Name: Vigyat Roll no : 10241500373 Class : 2O34

1. Class polygon contains data member width and height and public method set_value() to assign values to width and height. Class Rectangle and Triangle are inherited from polygon class. Both the classes contain public method calculate_area() to calculate the area of Rectangle and Triangle. Use base class pointer to access the derived class object and show the area calculated.

```
#include <iostream> using
namespace std;

class Polygon
{ protected:
   int width, height;

public:     void set_value(int
w, int h)
   {        width
= w;
height = h;
   }

   virtual void calculate_area() = 0;
};

class Rectangle : public Polygon
{ public:     void
calculate_area()
   {
      cout << "Area of Rectangle: " << width * height << endl;
   }
};

class Triangle : public Polygon
{
public:     void
calculate_area()
```

```
    {
        cout << "Area of Triangle: " << (width * height) / 2 << endl;
    }
};

int main()
{
    Polygon *p;

    Rectangle rect;
    Triangle tri;

    p = &rect;     p-
>set_value(30,10);     p-
>calculate_area();

    p = &tri;     p-
>set_value(100, 20);     p-
>calculate_area();

    return 0;
}
```



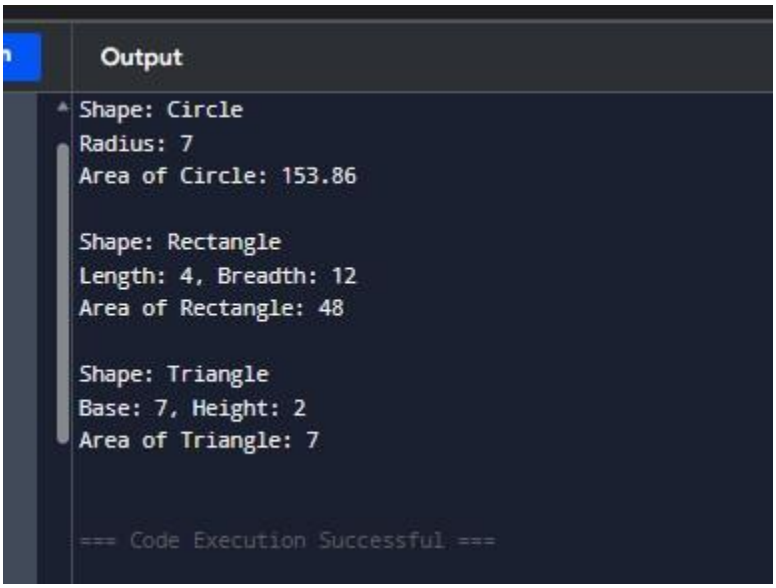
2. Write a program to create a class shape with functions area and display to find area and display the name of the shape and other essential component of the class. Create derived classes circle, rectangle and triangle each having overridden functions area and display. Write a program to find and display the area of circle, rectangle and triangle.

```
#include <iostream>
using namespace std;
```

```
class Shape
{ public:
   virtual void area() = 0;
virtual void display() = 0;
};

class Circle : public Shape
{     float
radius;

public:
Circle(float r)
   {         radius
= r;
   }

   void area()
   {
      cout << "Area of Circle: " << 3.14 * radius * radius << endl;
   }

   void display()
   {
      cout << "Shape: Circle" << endl;
cout << "Radius: " << radius << endl;
   }
};

class Rectangle : public Shape
{     float length,
breadth;

public:
   Rectangle(float l, float b)
   {         length
= l;         breadth
= b;     }

   void area()
   {
      cout << "Area of Rectangle: " << length * breadth << endl;
```

```
    }

    void display()
    {
        cout << "Shape: Rectangle" << endl;         cout << "Length: " <<
length << ", Breadth: " << breadth << endl;
    }
};

class Triangle : public Shape
{     float base,
height;

public:
    Triangle(float b, float h)
    {         base =
b;         height =
h;
    }

    void area()
    {
        cout << "Area of Triangle: " << 0.5 * base * height << endl;
    }

    void display()
    {
        cout << "Shape: Triangle" << endl;         cout << "Base: "
<< base << ", Height: " << height << endl;
    }
};

int main() {
    Shape *s;

    Circle c(7);
    Rectangle r(4,12);
    Triangle t(7, 2);

    s = &c;     s-
>display();     s-
```

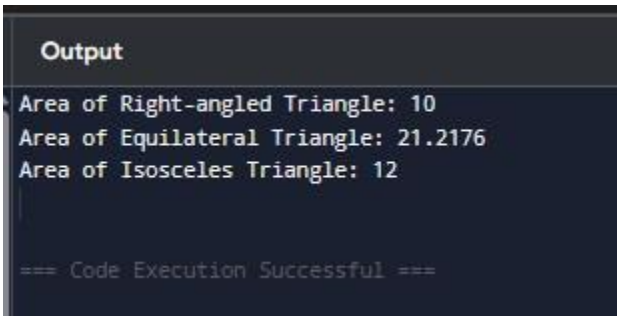
```
>area();     cout
<< endl;

   s = &r;     s-
>display();     s-
>area();     cout
<< endl;

   s = &t;     s-
>display();     s-
>area();

   return 0;
}
```



3. Write a C++ program to compute area of right angle triangle, equilateral triangle, isosceles triangle using function overloading concept.

```
#include <iostream>
#include <cmath> using
namespace std;

class Triangle
{ public:
```

```
    void area(float base, float height)
    {
        cout << "Area of Right-angled Triangle: " << 0.5 * base * height << endl;
    }

    void area(float side)
    {       float a = (sqrt(3) / 4) * side * side;       cout <<
"Area of Equilateral Triangle: " << a << endl;
    }

    void area(float equal_side, float base, int flag)
    {
        float height = sqrt(equal_side * equal_side - (base * base) / 4);
float a = 0.5 * base * height;       cout << "Area of Isosceles
Triangle: " << a << endl;
    }
};

int main() {
    Triangle t;

    t.area(4, 5);
    t.area(7);
    t.area(5, 6, 7);

    return 0;
}
```

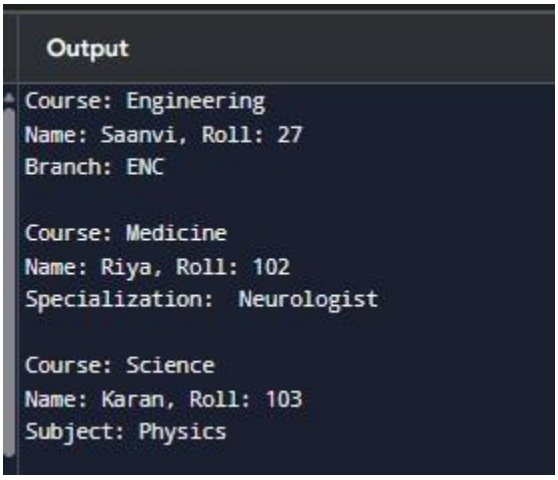


4. Write a program with Student as abstract class and create derive classes Engineering, Medicine and Science from base class Student. Create the objects of the derived classes and process them and access them using array of pointer of type base class Student.

```
#include <iostream> using
namespace std;

class Student
{ protected:
string name;
int roll;

public:
   Student(string n, int r)
   {        name
= n;        roll
= r;
   }

   virtual void display() = 0;
};

class Engineering : public Student
{    string
branch;

public:
   Engineering(string n, int r, string b) : Student(n, r)
   {
      branch = b;
   }

   void display()
   {
      cout << "Course: Engineering" << endl;         cout <<
"Name: " << name << ", Roll: " << roll << endl;         cout <<
"Branch: " << branch << endl << endl;
   }
};

class Medicine : public Student
{    string
specialization;
```

```
public:
   Medicine(string n, int r, string s) : Student(n, r)
   {        specialization
= s;
   }

   void display()
   {
      cout << "Course: Medicine" << endl;         cout << "Name: "
<< name << ", Roll: " << roll << endl;         cout <<
"Specialization: " << specialization << endl << endl;
   }
};

class Science : public Student
{    string
subject;

public:
   Science(string n, int r, string sub) : Student(n, r)
   {        subject =
sub;
   }

   void display()
   {
      cout << "Course: Science" << endl;         cout <<
"Name: " << name << ", Roll: " << roll << endl;         cout <<
"Subject: " << subject << endl << endl;
   }
};

int main()
{
   Student *s[3];

   s[0] = new Engineering("Saanvi", 27, "ENC");
s[1] = new Medicine("Riya", 102, " Neurologist");
s[2] = new Science("Karan", 103, "Physics");

   for (int i = 0; i < 3; i++)
```

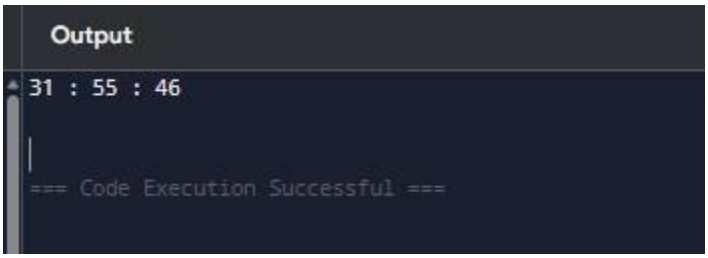
```
    {        s[i]-
>display();
    }

    return 0;
}
```



5. Create a class Time with three private variables int h,m,s; Create a function to overload '+' operator to add two time variables. int main(){
Time t1(5,15,34),t2(9,53,58),t3;
t3 = t1 + t2; t3.show();
}

```
#include <iostream> using
namespace std;

class Time {
int h, m, s;

public:
    Time() {}

    Time(int hh, int mm, int ss)
    {        h =
hh;        m =
mm;        s =
ss;
    }
```

```
    Time operator+(Time t)
    {
        Time temp;

        temp.s = s + t.s;         temp.m =
m + t.m + temp.s / 60;          temp.s =
temp.s % 60;

        temp.h = h + t.h + temp.m / 60;
temp.m = temp.m % 60;

        return temp;
    }

    void show()
    {
        cout << h << " : " << m << " : " << s << endl;
    }
};

int main()
{
    Time t1(11,2,17), t2(20,53,29), t3;
t3 = t1 + t2;     t3.show();
}
```

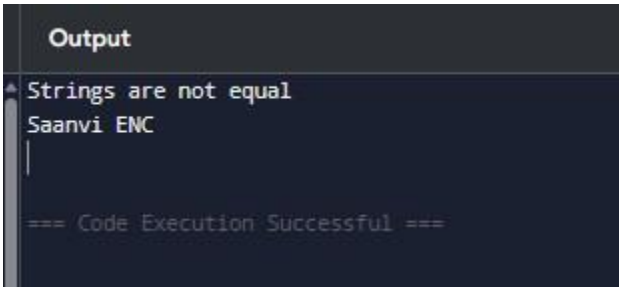


6. Define a class STRING and overload == to compare two strings and + operator for concatenation of two strings.

```
#include <iostream>
#include <cstring> using
namespace std;
```

```
class STRING
{    char
str[100];

public:
   STRING() {}

   STRING(const char s[])
   {
strcpy(str, s);
   }

   bool operator==(STRING s)
   {        return strcmp(str, s.str)
== 0;
   }

   STRING operator+(STRING s)
   {
      STRING temp;
strcpy(temp.str, str);
strcat(temp.str, s.str);        return
temp;
   }

   void display()
   {        cout << str <<
endl;
   }
};

int main()
{
   STRING s1("Saanvi "), s2("ENC"), s3;

   if (s1 == s2)        cout << "Strings are
equal" << endl;    else        cout << "Strings
are not equal" << endl;

   s3 = s1 + s2;
s3.display();
```

```
    return 0;
}
```



7. Write a program in C++ to create a class matrix and overload * operator using friend function to multiply two matrices.

```
#include <iostream>
using namespace std;

class Matrix
{
    int a[10][10], r, c;

public:
    Matrix() {}

    Matrix(int rows, int cols)
    {
        r = rows;
        c = cols;
}


    void input()
    {
        for (int i = 0; i < r; i++)
            for (int j = 0; j < c; j++)
```

```
                cin >> a[i][j];
}


    void display()
    {
        for (int i = 0; i < r; i++)
        {
            for (int j = 0; j < c; j++)
cout << a[i][j] << " ";
            cout << endl;
        }
    }

    friend Matrix operator*(Matrix m1, Matrix m2);
};

Matrix operator*(Matrix m1, Matrix m2)
{
    Matrix temp(m1.r, m2.c);

    for (int i = 0; i < m1.r; i++)
{
        for (int j = 0; j < m2.c; j++)
        {
            temp.a[i][j] = 0;
            for (int k = 0; k < m1.c; k++)
            {
                temp.a[i][j] += m1.a[i][k] * m2.a[k][j];
}
        }
    }

    return temp;
}
int main()
{
    Matrix m1(2, 2), m2(2, 2), m3;
```

```
    m1.input();
    m2.input();

    m3 = m1 * m2;

    m3.display();

    return 0;
}
```

```
19 22
43 50
PS C:\Users\abc\OneDrive\Desktop\Saanvi_1024150027\LAB
```

8. Overload to check array index out of bounds problem at run time.

```
#include <iostream>
using namespace std;

class Array
{
    int arr[5];

public:
    Array()
    {
        for (int i = 0; i < 5; i++)
            arr[i] = i * 10;
    }

    int& operator[](int index)
    {
```

```
        if (index < 0 || index >= 5)
        {
            cout << "Index out of bounds!" << endl;
            exit(0);
}
        return arr[index];
    }
};

int main()
{
    Array a;

    cout << a[2] << endl;
    cout << a[7] << endl;

    return 0;
}
```

```
Compilation terminated.
PS C:\Users\abc\OneDrive\Desktop\Saanvi_1024150027\LAB
PS C:\Users\abc\OneDrive\Desktop\Saanvi_1024150027\LAB
20
Index out of bounds!
PS C:\Users\abc\OneDrive\Desktop\Saanvi_1024150027\LAB
```

9. Overload '()' to input arbitrary number of input arguments for an object.

```
#include <iostream>
using namespace std;

class Numbers {

    int sum;

public:
    Numbers()
```

```
    {
        sum = 0;
}

    void operator()(int a)
    {
        sum += a;
    }

    void operator()(int a, int b)
    {
        sum += (a + b);
    }

    void operator()(int a, int b, int c)
    {
        sum += (a + b + c);
    }

    void display()
    {
        cout << "Sum: " << sum << endl;
    }
};

int main()
{
    Numbers n;

    n(10);
    n(20, 30);
    n(5, 5, 5);

    n.display();

    return 0;
```

```
}
```

```
PS C:\Users\abc\OneDrive\Desktop\S
Sum: 75
PS C:\Users\abc\OneDrive\Desktop\S
```

10. Write a program in C++ to overload input operator (>>) and output operator (<<).

```
#include <iostream>
using namespace std;

class Student
{
    int roll;
    string name;

public:
    friend istream& operator>>(istream& in, Student& s);
    friend ostream& operator<<(ostream& out, Student& s);
};

istream& operator>>(istream& in, Student& s) {

    cout << "Enter Roll Number: ";
    in >> s.roll;
    cout << "Enter Name: ";
    in >> s.name;
    return in;
}

ostream& operator<<(ostream& out, Student& s)
{
    out << "\nStudent Details:\n";
    out << "Roll: " << s.roll << endl;
    out << "Name: " << s.name << endl;
```

```
    return out; }



int main()
{
    Student s;

    cin >> s;
cout << s;

    return 0;
}
```

```
Roll: 4 Name: 1
PS C:\Users\abc\OneDrive\Desktop\Saanvi_1024150027\LAB 6> ./oops9
102
Saanvi
Roll: 102 Name: Saanvi
```

11. Write a program to convert basic data type (float) to user defined data type (object).

```
class Test {
private: //…
public:
Test(data_type) {
// conversion code
}
};
```

```cpp
#include <iostream>
using namespace std;

class Test
{
    float value;

public:
    Test(float x)
    {
        value = x;
    }

    void display()
    {
        cout << "Converted value: " << value << endl;
    }
};

int main()
{
    float f;

    cout << "Enter a float value: ";
    cin >> f;

    Test t = f;

    t.display();

    return 0;
}
```

```
PS C:\Users\abc\OneDrive\Desktop
Enter a float value: 5.87
Converted value: 5.87
PS C:\Users\abc\OneDrive\Desktop
```

12. Write a program to convert UDT to basic data type (float) class Test { public:
operator data_type() {
// conversion code
}
};

```cpp
#include <iostream>
using namespace std;

class Test
{
    float value;

public:
    Test(float x)
    {
        value = x;
}


    operator float()
    {
        return value;
    }
};

int main()
{
    Test t(7.5);

    float f;

    f = t;

    cout << "Converted value: " << f << endl;
```

```
    return 0;
}
```

```
PS C:\Users\abc\OneDrive\Desktop\Saanvi_1024150027\LAB 6> g++ oops11.cpp
PS C:\Users\abc\OneDrive\Desktop\Saanvi_1024150027\LAB 6> ./oops11
Converted value: 7.5
```

13. How will you convert one UDT to another UDT. For example conversion of polar to cartesian system. Polar p(10,5); Cartesian c = p; c.show();

```
#include <iostream>
#include <cmath>
using namespace std;

class Polar
{
    float r, angle;

public:
    Polar(float rad, float ang)
    {
        r = rad;
        angle = ang;
    }

    float getR()
    {
        return r;
    }

    float getAngle()
    {
        return angle;
    }
};

class Cartesian
```

```
{
    float x, y;

public:
    Cartesian() {}

    Cartesian(Polar p)
    {
        x = p.getR() * cos(p.getAngle());
        y = p.getR() * sin(p.getAngle());
    }

    void show()
    {
        cout << "x = " << x << ", y = " << y << endl;
    }
};

int main()
{
    Polar p(10, 0.5);

    Cartesian c;
    c = p;

    c.show();

    return 0;
}
```

```
PS C:\Users\abc\OneDrive\Desktop\
PS C:\Users\abc\OneDrive\Desktop\
x = 8.77583, y = 4.79426
PS C:\Users\abc\OneDrive\Desktop\
```